OAVS • TGT COMPUTER SCIENCE

MCQ Question Bank

Teacher Recruitment Examination — Odisha Adarsha Vidyalaya Sangathan

TOPIC COVERED

Decision and Case Control Structure

200	13	16
TOTAL MCQS	SECTIONS	QUESTION TYPES

Answers & Revision-Oriented Explanations Included for Every Question

Table of Contents

1. The if Statement	16 MCQs
2. The if-else Construct	17 MCQs
3. Logical Operators	18 MCQs
4. Compound Relational Tests	14 MCQs
5. Nested if Statements	15 MCQs
6. The else-if Construct	15 MCQs
7. Relational Operators	14 MCQs
8. Conditional Expression (Ternary) Operator	15 MCQs
9. The Switch Statement	17 MCQs
10. Switch Statement with break	14 MCQs
11. Switch Statement without break	13 MCQs
12. Concept of Case Label	16 MCQs
13. Mixed Review: Cross-Topic MCQs	16 MCQs

Strictly Syllabus-Bound: Every question in this bank is drawn exclusively from the Decision and Case Control Structure syllabus — if statement, if-else, logical operators, compound relational tests, nested if, else-if construct, relational operators, the conditional (ternary) operator, and the switch statement (with break, without break, and case labels). Loops, arrays, functions, pointers, and other advanced topics are intentionally excluded.

Section 1 of 13

The if Statement

Q1 **The if Statement** *Definition-Based*

What is the primary purpose of the if statement in C?

A. To repeat a block of code multiple times

B. To execute a block of code only when a specified condition is true

C. To define a function

D. To declare a variable

Ans: B

Explanation: The if statement is a decision-control structure that executes a block of statements only when the given condition evaluates to true (non-zero).

Q2 **The if Statement** *Syntax-Based*

Which of the following is the correct general syntax of a simple if statement in C?

A. if condition { statement; }

B. if (condition) statement;

C. if [condition] statement;

D. if condition: statement;

Ans: B

Explanation: The correct C syntax is if (condition) statement; — the condition must be enclosed in parentheses.

Q3 **The if Statement** *Conceptual*

In C, an if condition is considered true when its value is:

A. Zero

B. Negative only

C. Any non-zero value

D. A string value

Ans: C

Explanation: In C, any non-zero value (positive or negative) is treated as true; only zero is treated as false.

Q4 The if Statement *Working Mechanism*

If the condition in an if statement evaluates to false and no else part exists, what happens?

A. The program terminates

B. Control skips the if-block and moves to the next statement after it

C. An error is generated at runtime

D. The if-block executes anyway

Ans: B

Explanation: When the condition is false and there is no else, control simply skips the if-block and continues with the statement following it.

Q5 The if Statement *Flow of Execution*

What is the correct sequence of execution flow for a simple if statement?

A. Evaluate condition → execute block regardless → exit

B. Execute block → evaluate condition → exit

C. Evaluate condition → if true, execute block → continue with next statement

D. Skip condition → execute block → evaluate condition

Ans: C

Explanation: The condition is evaluated first; only if it is true does the block execute, after which control proceeds to the next statement.

Q6 The if Statement *Code-Based*

What will be the output of the following code?

```
int a = 5;
if (a > 10)
printf("Big");
printf("Done");
```

A. Big

B. Done

C. BigDone

D. No output

Ans: B

Explanation: Since a > 10 is false, printf("Big") is skipped, and only printf("Done") executes, so the output is Done.

Q7 The if Statement *Output-Based*

What is the output of the following code?

```
int x = 0;
if (x)
printf("True");
else
printf("False");
```

A. True
B. False
C. 0
D. Compilation error

Ans: B

Explanation: Since x is 0, the condition is false, so the else branch executes and False is printed.

Q8 The if Statement *Statement-Based*

Which statement about the if block containing multiple statements is TRUE?

A. Multiple statements after if never need braces
B. Multiple statements must be enclosed within curly braces { }
C. Only two statements are allowed inside if without braces
D. The compiler automatically adds braces for multiple statements

Ans: B

Explanation: Without braces, only the single statement immediately following if is part of the if-block; multiple statements require enclosing braces { }.

Q9 The if Statement *Tricky*

What is the output of the following code?

```
int x = 5;
if (x = 10)
printf("Yes");
else
printf("No");
```

A. Yes
B. No
C. Compilation error
D. 5

Ans: A

Explanation: The single = is an assignment, not comparison. x = 10 assigns 10 to x and the expression evaluates to 10 (non-zero, true), so Yes is printed. This tests the common == vs = confusion.

Q10 The if Statement *Applications*

Which real-life scenario is best modeled using a simple if statement?

A. Choosing between many mutually exclusive options

B. Repeating an action until a condition is met

C. Checking a single condition, such as whether a number is positive

D. Storing a list of values

Ans: C

Explanation: A simple if statement is ideal for checking one condition and taking action only when it holds true, such as testing if a number is positive.

Q11 The if Statement *Scenario-Based*

A program needs to print 'Eligible' only when age is 18 or above, and otherwise print nothing. Which construct is most suitable?

A. switch statement

B. for loop

C. simple if statement

D. while loop

Ans: C

Explanation: Since only one condition is checked and no action is needed for the false case, a simple if statement is the most efficient and suitable choice.

Q12 The if Statement *Fill-in-the-Blank*

The condition of an if statement in C must always be enclosed within _____.

A. square brackets [ ]

B. curly braces { }

C. parentheses ( )

D. angle brackets < >

Ans: C

Explanation: In C syntax, the condition of an if statement is mandatorily written inside parentheses ( ).

Q13 The if Statement *Assertion-Reason*

Assertion (A): The if statement is called a one-way decision structure.
Reason (R): It executes a block of code only when the condition is true and takes no action otherwise.
Choose the correct option:

A. Both A and R are true, and R is the correct explanation of A

B. Both A and R are true, but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: A

Explanation: The if statement offers only one path of action (when true) with no alternative branch, correctly justifying why it is called one-way decision-making.

Q14 The if Statement *Difficult*

What is the output of the following nested single-statement if code (without braces)?

```
int a = 1, b = 0;
if (a)
if (b)
printf("X");
else
printf("Y");
```

A. X

B. Y

C. No output

D. Compilation error

Ans: B

Explanation: Due to the dangling-else rule, the else binds to the nearest unmatched if (the inner if(b)). Since a is true, the inner if(b) is tested; b is 0 (false), so else executes, printing Y.

Q15 The if Statement *Previous-Year Pattern*

Which of the following is NOT a valid condition inside an if statement in C?

A. if (a == b)

B. if (a > 0 && b > 0)

C. if (a; b)

D. if (a)

Ans: C

Explanation: if (a; b) is invalid syntax because a semicolon cannot appear inside the condition parentheses of an if statement; this causes a compilation error.

Q16 The if Statement *Comparison-Based*

How does a simple if statement differ from an if-else statement?

A. if has two branches, if-else has one

B. if provides action only for the true case; if-else provides actions for both true and false cases

C. They are functionally identical

D. if-else cannot use relational operators

Ans: B

Explanation: A simple if executes code only when the condition is true, offering no alternative, whereas if-else defines separate actions for both true and false outcomes.

Section 2 of 13

The if-else Construct

Q17 | **The if-else Construct** | *Definition-Based*

What is the main advantage of if-else over a simple if statement?

A. It executes faster

B. It provides an alternative block of code to execute when the condition is false

C. It removes the need for relational operators

D. It can only be used with loops

Ans: B

Explanation: if-else provides a two-way decision: one block runs when the condition is true and another (else block) runs when it is false.

Q18 | **The if-else Construct** | *Syntax-Based*

Which is the correct syntax for the if-else construct?

A. if (condition) statement1; else statement2;

B. if condition then statement1 else statement2

C. if (condition) { statement1 } else { statement2 } end

D. if-else (condition) statement1 statement2

Ans: A

Explanation: The correct C syntax is: if (condition) statement1; else statement2; — else must directly follow the if block without any condition of its own.

Q19 | **The if-else Construct** | *Working*

In an if-else statement, when is the else block executed?

A. Always, regardless of the condition

B. Only when the if condition evaluates to false

C. Only when the if condition evaluates to true

D. Never, it is just a placeholder

Ans: B

Explanation: The else block executes only when the associated if condition evaluates to false (zero).

Q20 The if-else Construct *Decision Making*

How many blocks of code can actually execute in a single run of an if-else statement?

A. Both blocks always execute

B. Neither block executes

C. Exactly one block executes: either if-block or else-block, never both

D. It depends on the compiler

Ans: C

Explanation: if-else is mutually exclusive: exactly one of the two blocks executes per run, based on whether the condition is true or false.

Q21 The if-else Construct *Program Flow*

What is the output of the following code?

```
int marks = 40;
if (marks >= 50)
printf("Pass");
else
printf("Fail");
```

A. Pass

B. Fail

C. PassFail

D. No output

Ans: B

Explanation: Since marks (40) is not >= 50, the condition is false, so the else block executes and Fail is printed.

Q22 The if-else Construct *Applications*

Which scenario is best suited for an if-else construct rather than a plain if?

A. Printing a message only if a file exists

B. Determining whether a number is even or odd, with output required for both cases

C. Logging an error only when it occurs

D. Incrementing a counter unconditionally

Ans: B

Explanation: When distinct output/action is needed for both possible outcomes (even and odd), if-else is appropriate since it defines behavior for both true and false cases.

Q23 The if-else Construct *Differences from Simple if*

Which of the following best distinguishes if-else from a simple if?

A. if-else cannot use logical operators

B. if-else always requires nested statements

C. Simple if handles only the true case; if-else handles both true and false cases

D. There is no difference; they behave identically

Ans: C

Explanation: A simple if only reacts when the condition is true, while if-else explicitly defines an alternate action for the false case as well.

Q24 The if-else Construct *Code-Based*

What is the output of the following code?

```
int n = -5;
if (n >= 0)
printf("Non-negative");
else
printf("Negative");
```

A. Non-negative

B. Negative

C. -5

D. Compilation error

Ans: B

Explanation: n is -5, so n >= 0 is false; hence the else branch executes and Negative is printed.

Q25 The if-else Construct *Output-Based*

What is the output of the following code?

```
int a = 7;
if (a % 2 == 0)
printf("Even");
else
printf("Odd");
```

A. Even

B. Odd

C. 7

D. 0

Ans: B

Explanation: 7 % 2 evaluates to 1, which is not equal to 0, so the condition is false and Odd is printed via the else branch.

Q26 The if-else Construct *Tricky*

What is a common syntax error made when writing an if-else statement?

A. Forgetting to enclose the condition in parentheses

B. Placing a semicolon immediately after the if condition, e.g. if (a > b); else ...

C. Both A and B are common errors

D. There is no possible syntax error in if-else

Ans: C

Explanation: Both omitting parentheses around the condition and mistakenly placing a semicolon right after if(condition) — which creates an empty statement and detaches else — are common beginner errors.

Q27 The if-else Construct *Fill-in-the-Blank*

In C, the else keyword must always be paired with a preceding _____ statement.

A. while

B. if

C. switch

D. for

Ans: B

Explanation: else has no independent meaning in C; it must always be associated with a preceding if statement.

Q28 The if-else Construct *Assertion-Reason*

Assertion (A): if-else is called a two-way branching structure.
Reason (R): It allows only one of the two alternative blocks to execute depending on the condition's truth value.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: The name 'two-way' reflects that exactly one of the two mutually exclusive paths executes, which is precisely what R describes.

Q29 The if-else Construct *Comparison-Based*

Which statement correctly compares if-else with switch for a simple two-outcome decision?

A. switch is always faster for two outcomes

B. if-else is generally simpler and more direct for a two-outcome (binary) decision

C. if-else cannot handle two outcomes

D. switch and if-else are identical in every case

Ans: B

Explanation: For a simple binary decision, if-else is more direct and readable than a switch statement, which is better suited to multiple discrete cases.

Q30 The if-else Construct *Scenario-Based*

A shop gives a discount if the purchase amount exceeds ₹1000, otherwise no discount is given but a message must still be shown. Which is most appropriate?

A. Simple if statement only

B. if-else statement

C. Nested loop

D. Switch without break

Ans: B

Explanation: Since a message must be displayed in both cases (discount and no discount), if-else is appropriate as it defines behavior for both branches.

Q31 The if-else Construct *Difficult*

What is the output of the following code?

```
int x = 5, y = 5;
if (x == y)
if (x > 10)
printf("A");
else
printf("B");
else
printf("C");
```

A. A

B. B

C. C

D. No output

Ans: B

Explanation: x == y is true, so the outer if executes the inner if-else. x > 10 is false (5 is not > 10), so the inner else runs, printing B.

Q32 The if-else Construct *Previous-Year Pattern*

Which of the following if-else statements will always execute the else part regardless of variable values?

A. if (1) ... else ...

B. if (0) ... else ...

C. if (a == a) ... else ...

D. if (a >= 0 || a < 0) ... else ...

Ans: B

Explanation: if (0) has a condition that is always false (0 is falsy in C), so the else block will always execute.

Q33 The if-else Construct *Statement-Based*

Which statement about dangling else is TRUE?

A. An else always pairs with the first if in the program

B. An else pairs with the nearest preceding unmatched if, unless braces dictate otherwise

C. Dangling else causes a compilation error in C

D. C does not allow nested if without else

Ans: B

Explanation: The dangling-else ambiguity is resolved in C by pairing an else with the nearest unmatched if, unless curly braces explicitly indicate a different pairing.

Section 3 of 13

Logical Operators

Q34 Logical Operators *Definition-Based*

Which symbol represents the logical AND operator in C?

A. &
B. &&
C. AND
D. |

Ans: B

Explanation: && is the logical AND operator in C, which returns true only when both operands are true. A single & is the bitwise AND.

Q35 Logical Operators *Definition-Based*

Which symbol represents the logical OR operator in C?

A. |
B. ||
C. OR
D. &&

Ans: B

Explanation: || is the logical OR operator, returning true when at least one operand is true. A single | is the bitwise OR.

Q36 Logical Operators *Definition-Based*

Which symbol represents the logical NOT operator in C?

A. ~
B. !
C. not
D. ^

Ans: B

Explanation: ! is the logical NOT operator that reverses the truth value of its operand; ~ is the bitwise complement, not logical NOT.

Q37 Logical Operators *Truth Table*

For the AND (&&) operator, when is the result true?

A. When at least one operand is true
B. When both operands are true
C. When both operands are false
D. When exactly one operand is true

Ans: B

Explanation: The AND (&&) operator yields true only when both operand expressions evaluate to true (non-zero).

Q38 Logical Operators *Truth Table*

For the OR (||) operator, when is the result false?

A. When both operands are true
B. When at least one operand is true
C. Only when both operands are false
D. When either operand is false

Ans: C

Explanation: The OR (||) operator returns false only in the single case where both operands evaluate to false (zero).

Q39 Logical Operators *Conceptual*

What does the NOT (!) operator do to a non-zero value?

A. Leaves it unchanged
B. Converts it to 0 (false)
C. Converts it to -1
D. Causes a runtime error

Ans: B

Explanation: The ! operator inverts truth values: applying ! to any non-zero (true) value produces 0 (false); applying it to 0 produces 1 (true).

Q40 Logical Operators *Usage in Decision Making*

Which expression correctly checks if a number x lies between 10 and 20 (inclusive)?

A. if (10 <= x <= 20)
B. if (x >= 10 && x <= 20)
C. if (x >= 10 || x <= 20)
D. if (x >= 10, x <= 20)

Ans: B

Explanation: C does not support chained relational expressions like 10 <= x <= 20 the way mathematics does; the correct approach is combining two conditions with &&.

Q41 Logical Operators *Evaluation of Expressions*

What is the result of the expression (5 > 3) && (2 > 4)?

A. 1 (true)
B. 0 (false)
C. -1
D. Compilation error

Ans: B

Explanation: (5 > 3) is true (1) but (2 > 4) is false (0). Since AND requires both to be true, the overall result is 0 (false).

Q42 Logical Operators *Evaluation of Expressions*

What is the result of the expression (0) || (7 > 2)?

A. 0
B. 1
C. 7
D. Compilation error

Ans: B

Explanation: 0 is false, but (7 > 2) is true; since OR needs only one operand to be true, the result is 1 (true).

Q43 Logical Operators *Code-Based*

What is the output of the following code?

```
int a = 5, b = 0;
if (a && b)
printf("Yes");
else
printf("No");
```

A. Yes
B. No
C. 5
D. 0

Ans: B

Explanation: a is 5 (true) but b is 0 (false); since && requires both to be true, the condition is false, so No is printed.

Q44 Logical Operators *Output-Based*

What is the output of the following code?

```
int a = 0, b = 0;
if (!a && !b)
printf("Both false");
else
printf("At least one true");
```

A. Both false

B. At least one true

C. 0

D. Compilation error

Ans: A

Explanation: !a becomes true (since a=0) and !b becomes true (since b=0); true && true is true, so 'Both false' is printed.

Q45 Logical Operators *Tricky*

In the expression (a++ > 0) && (b++ > 0), if a = -1, what happens to b due to short-circuit evaluation?

A. b is incremented regardless

B. b is never evaluated or incremented, because the left operand is false

C. b causes a compilation error

D. b is decremented

Ans: B

Explanation: C uses short-circuit evaluation for &&: if the left operand is false, the right operand is not evaluated at all, so b++ never executes.

Q46 Logical Operators *Tricky*

In the expression (a > 0) || (b++ > 0), if a = 5, what happens to b due to short-circuit evaluation?

A. b is incremented

B. b is not evaluated because the left operand is already true

C. b causes an error

D. b is set to 0

Ans: B

Explanation: For ||, if the left operand is true, C short-circuits and skips evaluating the right operand entirely, so b++ does not execute.

Q47 Logical Operators *Applications*

Logical operators are most useful for:

A. Performing arithmetic calculations

B. Combining multiple conditions into a single decision-making expression

C. Declaring variables

D. Formatting output

Ans: B

Explanation: Logical operators (&&, ||, !) allow multiple relational conditions to be combined into a single compound expression for decision-making.

Q48 Logical Operators *Match-the-Following*

Match the operator with its correct description:
(1) && (2) || (3) !
(a) Negates a condition (b) True if both are true (c) True if either is true

A. 1-a, 2-b, 3-c

B. 1-b, 2-c, 3-a

C. 1-c, 2-a, 3-b

D. 1-b, 2-a, 3-c

Ans: B

Explanation: && matches 'true if both are true' (b), || matches 'true if either is true' (c), and ! matches 'negates a condition' (a).

Q49 Logical Operators *Comparison-Based*

What is the key difference between && and || in terms of decision requirement?

A. && requires only one operand true, || requires all operands true

B. && requires all operands true, || requires at least one operand true

C. They behave identically

D. && is used only with integers, || only with floats

Ans: B

Explanation: && demands every operand be true for the result to be true, whereas || only needs a single true operand for the result to be true.

Q50 Logical Operators *Difficult*

What is the output of: printf("%d", !(5 > 3) || (2 == 2));

A. 0

B. 1

C. 2

D. Compilation error

Ans: B

Explanation: (5>3) is true, so !(5>3) is 0 (false). (2==2) is true (1). false || true evaluates to true, so the output is 1.

Q51 Logical Operators *Previous-Year Pattern*

Which logical operator has the lowest precedence among &&, ||, and !?

A. !

B. &&

C. ||

D. All have equal precedence

Ans: C

Explanation: In C's operator precedence, ! (NOT) has the highest precedence, followed by && (AND), with || (OR) having the lowest precedence among the three.

Section 4 of 13

Compound Relational Tests

Q52 Compound Relational Tests *Definition-Based*

What is a compound relational test in C?

A. A single relational expression using one operator

B. A combination of two or more relational expressions joined by logical operators

C. A test that uses only arithmetic operators

D. A test performed inside a loop only

Ans: B

Explanation: A compound relational test combines multiple relational expressions (using <, >, ==, etc.) with logical operators (&&, ||) to form a single complex condition.

Q53 Compound Relational Tests *Combining Relational Expressions*

Which expression correctly checks if a year is a leap year using compound relational tests (simplified condition, ignoring century rule)?

A. year % 4 == 0

B. year % 4 == 0 && year % 100 != 0

C. year % 4 == 0 || year % 100 == 0

D. year % 4 != 0 && year % 100 == 0

Ans: B

Explanation: A basic leap year check combines two relational tests with &&: divisible by 4 AND not divisible by 100 (the full rule also OR's divisibility by 400, but among the given options this is the correctly combined form).

Q54 Compound Relational Tests *Use with Logical Operators*

Which compound condition checks whether a character ch is an uppercase letter?

A. ch >= 'A' || ch <= 'Z'

B. ch >= 'A' && ch <= 'Z'

C. ch == 'A' && ch == 'Z'

D. ch >= 'a' && ch <= 'z'

Ans: B

Explanation: To check a range, both boundary conditions must hold simultaneously, so && is used: ch must be greater than or equal to 'A' AND less than or equal to 'Z'.

Q55 Compound Relational Tests *Expression Evaluation*

Evaluate: (10 > 5) && (10 < 20) && (10 != 15)

A. 0 (false)
B. 1 (true)
C. 10
D. Compilation error

Ans: B

Explanation: All three relational sub-expressions are true: 10>5 (true), 10<20 (true), 10!=15 (true). Since all are ANDed, the overall result is true (1).

Q56 Compound Relational Tests *Expression Evaluation*

Evaluate: (a == 5 || a == 10) when a = 7

A. 1 (true)
B. 0 (false)
C. 7
D. 5

Ans: B

Explanation: Neither a==5 nor a==10 is true when a=7, so with OR both being false means the overall expression is false (0).

Q57 Compound Relational Tests *Applications*

Compound relational tests are especially useful for:

A. Declaring arrays
B. Checking range-based or multi-criteria conditions in a single if statement
C. Defining functions
D. Performing input/output operations

Ans: B

Explanation: Compound tests let a program validate multiple criteria at once — such as range checks or multi-value comparisons — within a single decision statement.

Q58 Compound Relational Tests *Code-Based*

What is the output of the following code?

```
int age = 25;
if (age >= 18 && age <= 60)
printf("Working age");
else
printf("Not working age");
```

A. Working age
B. Not working age
C. 25
D. Compilation error

Ans: A

Explanation: age (25) satisfies both age>=18 and age<=60, so the compound AND condition is true, printing 'Working age'.

Q59 Compound Relational Tests *Output-Based*

What is the output of the following code?

```
int marks = 35;
if (marks < 33 || marks > 90)
printf("Boundary case");
else
printf("Normal range");
```

A. Boundary case
B. Normal range
C. 35
D. Error

Ans: B

Explanation: marks(35) is neither less than 33 nor greater than 90, so both OR conditions are false, making the else branch execute: 'Normal range'.

Q60 Compound Relational Tests *Scenario-Based*

A password is considered valid only if it is at least 8 characters long AND contains at least one digit. Which structure best expresses this in a compound condition?

A. length >= 8 || hasDigit
B. length >= 8 && hasDigit
C. length < 8 && hasDigit
D. length == 8 || hasDigit

Ans: B

Explanation: Since both criteria (length and digit presence) must be simultaneously satisfied, the correct combination uses && (AND).

Q61 Compound Relational Tests *Fill-in-the-Blank*

To check whether a number is either negative or greater than 100, the correct compound test uses the _____ operator.

A. &&
B. ||
C. !
D. %

Ans: B

Explanation: Since only one of the two conditions ('negative' OR 'greater than 100') needs to be true, the || (OR) operator is the correct choice.

Q62 Compound Relational Tests *Tricky*

What is wrong with the expression: if (5 < x < 10) to check if x lies between 5 and 10?

A. Nothing, it works correctly in C
B. It is evaluated left to right: (5<x) yields 0 or 1, which is then compared with 10, producing incorrect results
C. It causes a compilation error
D. It only works for floating-point numbers

Ans: B

Explanation: C evaluates 5 < x < 10 as (5 < x) < 10. Since (5<x) yields 0 or 1, and both 0<10 and 1<10 are true, the expression is always true regardless of x's actual value — a classic beginner mistake.

Q63 Compound Relational Tests *Comparison-Based*

How does a compound relational test differ from a simple relational test?

A. A compound test uses only one condition
B. A compound test combines multiple relational conditions using logical operators, while a simple test checks only one condition
C. There is no functional difference
D. Compound tests cannot be used in if statements

Ans: B

Explanation: A simple relational test evaluates a single comparison, whereas a compound test merges two or more such comparisons using logical operators.

Q64 Compound Relational Tests *Difficult*

What is the output of the following code?

```
int x = 15;
if (x > 10 && x < 20 && x % 5 == 0)
printf("Match");
else
printf("No Match");
```

A. Match

B. No Match

C. 15

D. 5

Ans: A

Explanation: x(15) satisfies x>10, x<20, and x%5==0 (15%5=0) — all three conditions are true, so the AND chain is true and 'Match' prints.

Q65 Compound Relational Tests *Previous-Year Pattern*

Which compound condition correctly validates that a character is a vowel (uppercase or lowercase 'a','e','i','o','u')?

A. ch == 'a' && ch == 'e' && ch == 'i'

B. ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||ch=='A'||ch=='E'||ch=='I'||ch=='O'||ch=='U'

C. ch >= 'a' && ch <= 'u'

D. ch != 'a' || ch != 'e'

Ans: B

Explanation: Checking membership in a discrete set of values (vowels) requires ORing individual equality tests, since the values are not a contiguous range.

Section 5 of 13

Nested if Statements

Q66 Nested if Statements *Definition-Based*

What is a nested if statement?

A. An if statement placed inside a loop

B. An if or if-else statement placed inside another if or else block

C. Two if statements placed one after another at the same level

D. An if statement with multiple conditions joined by &&

Ans: B

Explanation: A nested if occurs when one if (or if-else) statement is written entirely inside the body of another if or else block, allowing multi-level decision making.

Q67 Nested if Statements *Syntax-Based*

Which of the following correctly represents nested if syntax?

A. if (cond1) if (cond2) statement;

B. if (cond1) && if (cond2) statement;

C. if (cond1, cond2) statement;

D. if (cond1) then if (cond2) statement;

Ans: A

Explanation: In nested if, an inner if statement is placed directly inside the body of an outer if, without any special connecting syntax.

Q68 Nested if Statements *Execution Flow*

In a nested if structure, when does the inner if get evaluated?

A. Always, regardless of the outer condition

B. Only if the outer if condition is true

C. Only if the outer if condition is false

D. Simultaneously with the outer condition

Ans: B

Explanation: The inner if is only reached and evaluated when the outer if's condition evaluates to true; otherwise, the inner if is skipped entirely.

Q69 Nested if Statements *Code-Based*

What is the output of the following code?

```
int a = 10, b = 20;
if (a > 5) {
if (b > 15)
printf("Both conditions true");
}
```

A. Both conditions true

B. No output

C. a>5

D. Compilation error

Ans: A

Explanation: a>5 (10>5) is true, so the inner if executes; b>15 (20>15) is also true, so 'Both conditions true' is printed.

Q70 Nested if Statements *Output-Based*

What is the output of the following code?

```
int x = 3;
if (x > 0) {
if (x > 10)
printf("Large");
else
printf("Small positive");
} else {
printf("Non-positive");
}
```

A. Large

B. Small positive

C. Non-positive

D. No output

Ans: B

Explanation: x>0 (3>0) is true, entering the outer if block. Inside, x>10 is false (3 is not >10), so the inner else runs, printing 'Small positive'.

Q71 Nested if Statements *Applications*

Nested if statements are most appropriate when:

A. Only a single condition needs checking

B. A decision depends on another decision, i.e., conditions are hierarchical

C. Repeating a task multiple times

D. Declaring multiple variables

Ans: B

Explanation: Nested if is ideal when a secondary condition only makes sense to check after a primary condition has already been satisfied — a hierarchical decision structure.

Q72 Nested if Statements *Advantages*

What is an advantage of nested if over a single compound condition using &&?

A. Nested if is always faster to execute

B. Nested if allows different actions at each level and can express more complex hierarchical logic clearly

C. Nested if requires fewer lines of code

D. Nested if cannot use else

Ans: B

Explanation: Nested if structures allow separate handling and distinct actions at each decision level, making complex hierarchical logic more explicit and manageable than a single compound condition.

Q73 Nested if Statements *Program Analysis*

In the following code, which output is produced for a = 15, b = 5?

```
if (a > 10) {
if (b > 10)
printf("Case1");
else
printf("Case2");
} else {
printf("Case3");
}
```

A. Case1

B. Case2

C. Case3

D. No output

Ans: B

Explanation: a>10 (15>10) is true, entering the outer block. b>10 (5>10) is false, so the inner else executes, printing 'Case2'.

Q74 Nested if Statements *Scenario-Based*

A grading system first checks if a student passed, and only if passed, checks whether marks qualify for distinction. Which structure fits best?

A. Simple if statement

B. Nested if statement

C. Switch without break

D. A single relational operator

Ans: B

Explanation: Since the distinction check is only meaningful after confirming a pass, this hierarchical dependency is best modeled using nested if statements.

Q75 Nested if Statements *Tricky*

What potential issue can arise with deeply nested if statements without proper braces?

A. They automatically convert to switch statements

B. Dangling-else ambiguity, where else may bind to an unintended if

C. The program refuses to compile

D. Nested if cannot use relational operators

Ans: B

Explanation: Without explicit braces, an else can mistakenly bind to the nearest inner if instead of the intended outer if, leading to unexpected program logic (dangling-else problem).

Q76 Nested if Statements *Comparison-Based*

How does nested if differ from the else-if ladder?

A. They are exactly the same construct

B. Nested if places conditions inside one another (hierarchical), while else-if ladder checks a sequence of alternative conditions at the same level

C. Nested if cannot include an else

D. else-if ladder cannot be replaced by nested if

Ans: B

Explanation: Nested if embeds one decision within another (parent-child relationship), whereas else-if ladder tests a series of independent alternatives one after another at the same logical level.

Q77 Nested if Statements *Fill-in-the-Blank*

In nested if statements, the inner if is executed only when the _____ condition is satisfied.

A. global
B. outer
C. final
D. unrelated

Ans: B

Explanation: The inner if is reached and evaluated only after the outer (enclosing) if condition has been satisfied (evaluated true).

Q78 Nested if Statements *Difficult*

What is the output of the following triple-nested code for a=5, b=5, c=5?

```
if (a == b) {
if (b == c) {
if (a == c)
printf("All equal");
else
printf("a!=c");
} else
printf("b!=c");
} else
printf("a!=b");
```

A. All equal
B. a!=c
C. b!=c
D. a!=b

Ans: A

Explanation: a==b is true (5==5), b==c is true (5==5), a==c is true (5==5), so the innermost printf executes, printing 'All equal'.

Q79 Nested if Statements *Assertion-Reason*

Assertion (A): Nested if statements can replace certain uses of compound relational tests.
Reason (R): Both approaches allow multiple conditions to influence a single decision outcome.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: Nested if and compound relational tests (with &&) can often achieve equivalent logic since both evaluate multiple conditions to determine the final action, validating R as the explanation for A.

Q80 Nested if Statements *Previous-Year Pattern*

Which of these best explains why excessive nesting of if statements is generally discouraged in good programming practice?

A. It causes syntax errors

B. It reduces code readability and increases maintenance difficulty

C. It is not supported by the C compiler

D. It automatically slows down compilation infinitely

Ans: B

Explanation: While syntactically valid, deeply nested if statements make code harder to read, debug, and maintain, which is why alternatives like else-if ladders or switch are often preferred for many-branch logic.

Section 6 of 13

The else-if Construct

Q81 | **The else-if Construct** | *Definition-Based*

What is the else-if ladder used for?

A. Repeating a block of statements

B. Testing a series of conditions in sequence and executing the block for the first true condition

C. Declaring multiple variables at once

D. Performing only arithmetic operations

Ans: B

Explanation: The else-if ladder allows a program to check multiple alternative conditions in sequence, executing the block associated with the first condition found true.

Q82 | **The else-if Construct** | *Syntax-Based*

Which is the correct syntax pattern for an else-if ladder?

A. if (c1) s1; elseif (c2) s2; else s3;

B. if (c1) s1; else if (c2) s2; else s3;

C. if (c1) s1; else (c2) s2; else s3;

D. if (c1, c2, c3) s1; s2; s3;

Ans: B

Explanation: The correct C syntax chains conditions using 'else if', ending with an optional final 'else' for the default case: if(c1) s1; else if(c2) s2; else s3;

Q83 | **The else-if Construct** | *Ladder Structure*

In an else-if ladder, once a condition is found true and its block executes, what happens next?

A. All remaining conditions are still checked

B. The remaining else-if and else branches are skipped

C. The program restarts from the first condition

D. An error occurs

Ans: B

Explanation: As soon as one condition in the ladder evaluates true and its block runs, the rest of the ladder is bypassed entirely — no further conditions are tested.

Q84 The else-if Construct *Execution Sequence*

In an else-if ladder, conditions are evaluated:

A. In random order

B. From bottom to top

C. Sequentially from top to bottom until one is true

D. All simultaneously

Ans: C

Explanation: Conditions in an else-if ladder are tested one after another, from the top of the ladder downward, stopping at the first true condition.

Q85 The else-if Construct *Code-Based*

What is the output of the following code?

```
int marks = 75;
if (marks >= 90)
printf("A");
else if (marks >= 75)
printf("B");
else if (marks >= 60)
printf("C");
else
printf("D");
```

A. A

B. B

C. C

D. D

Ans: B

Explanation: marks(75) fails marks>=90, but satisfies marks>=75, so 'B' is printed, and the remaining conditions are skipped.

Q86 | **The else-if Construct** | *Output-Based*

What is the output of the following code?

```
int num = -5;
if (num > 0)
printf("Positive");
else if (num < 0)
printf("Negative");
else
printf("Zero");
```

A. Positive

B. Negative

C. Zero

D. No output

Ans: B

Explanation: num>0 is false (-5 is not > 0), but num<0 is true, so 'Negative' is printed.

Q87 | **The else-if Construct** | *Decision Hierarchy*

In an else-if ladder used for grading (A, B, C, D), what determines the correct order of conditions?

A. Alphabetical order of grade letters

B. Conditions should typically be ordered from the most restrictive/highest threshold to the least, ensuring correct classification

C. Order does not matter at all

D. Conditions must be in random order for correctness

Ans: B

Explanation: Because the ladder stops at the first true condition, thresholds must be ordered correctly (e.g., highest marks first) to avoid misclassifying values that satisfy multiple conditions.

Q88 | **The else-if Construct** | *Applications*

The else-if ladder is best suited for:

A. Checking a single fixed condition

B. Selecting one action from several mutually exclusive, ranked conditions

C. Repeating an operation indefinitely

D. Declaring constants

Ans: B

Explanation: else-if ladders excel at scenarios with multiple mutually exclusive alternatives that need to be tested and ranked, such as grade classification or range-based categorization.

Q89 The else-if Construct *Comparisons with Nested if*

How does else-if ladder differ structurally from nested if?

A. else-if ladder tests independent alternative conditions at the same conceptual level; nested if tests conditions dependent on a prior condition being true

B. They are functionally and structurally identical

C. else-if ladder cannot use relational operators

D. Nested if always executes faster

Ans: A

Explanation: In an else-if ladder, each condition is an independent alternative to the others, whereas in nested if, the inner condition is only relevant because the outer condition already succeeded.

Q90 The else-if Construct *Scenario-Based*

A program must classify a person's age group as Child, Teen, Adult, or Senior, based on distinct age ranges. Which construct is most suitable?

A. Simple if statement

B. else-if ladder

C. Compound relational test alone

D. A single logical NOT expression

Ans: B

Explanation: Since there are multiple mutually exclusive categories based on ranked ranges, the else-if ladder is the natural and readable choice.

Q91 The else-if Construct *Tricky*

What is the output of the following code, given num = 50?

```
if (num > 10)
printf("A");
else if (num > 30)
printf("B");
else if (num > 40)
printf("C");
```

A. A

B. B

C. C

D. No output

Ans: A

Explanation: Even though num(50) also satisfies num>30 and num>40, the ladder stops at the FIRST true condition (num>10), so 'A' is printed — a common logic error from incorrect condition ordering.

Q92 The else-if Construct *Fill-in-the-Blank*

In an else-if ladder, the final else block (if present) executes when _____.

A. all preceding conditions are true
B. none of the preceding conditions are true
C. the first condition is true
D. the program always executes it first

Ans: B

Explanation: The trailing else in a ladder acts as the default/catch-all case, executing only when none of the preceding conditions evaluated to true.

Q93 The else-if Construct *Assertion-Reason*

Assertion (A): The else-if ladder is more efficient than writing multiple independent if statements for mutually exclusive conditions.
Reason (R): Once a true condition is found in the ladder, remaining condition checks are skipped, saving unnecessary evaluations.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. Both A and R are true, but R does not explain A
C. A is true, R is false
D. A is false, R is true

Ans: A

Explanation: Independent if statements would each be evaluated regardless of prior results, whereas the else-if ladder short-circuits further checks after the first true condition, which correctly explains the efficiency gain.

Q94 The else-if Construct *Difficult*

What is the output for x = 0 in the following?

```
if (x > 0)
printf("Positive");
else if (x == 0)
printf("Zero");
else
printf("Negative");
```

A. Positive
B. Zero
C. Negative
D. No output

Ans: B

Explanation: x>0 is false (0 is not >0), so the ladder proceeds; x==0 is true, so 'Zero' is printed, skipping the final else.

Q95 The else-if Construct *Previous-Year Pattern*

Which of the following is a valid reason to prefer an else-if ladder over a switch statement?

A. The conditions involve ranges or relational comparisons rather than exact matches on a single variable

B. The ladder is always faster than switch in every case

C. switch cannot be used at all in C

D. else-if cannot use variables

Ans: A

Explanation: switch only tests exact equality of an integer/character expression against constants, while else-if ladders can evaluate range-based or complex relational conditions, making them preferable in such cases.

Section 7 of 13

Relational Operators

Q96 Relational Operators *Definition-Based*

What is the purpose of relational operators in C?

A. To perform arithmetic operations

B. To compare two values and produce a true/false (1/0) result

C. To allocate memory dynamically

D. To declare data types

Ans: B

Explanation: Relational operators compare two operands and yield a Boolean-like result: 1 if the comparison is true, 0 if false.

Q97 Relational Operators *Symbols*

Which operator checks if two values are exactly equal in C?

A. =

B. ==

C. !=

D. <>

Ans: B

Explanation: == is the equality relational operator in C. A single = is the assignment operator, a frequent source of confusion for beginners.

Q98 Relational Operators *Symbols*

Which operator represents 'not equal to' in C?

A. <>

B. !=

C. !==

D. ><

Ans: B

Explanation: != is the correct C operator for 'not equal to'. <> is not valid C syntax (it is used in some other languages like Pascal or SQL).

Q99 Relational Operators *Meaning*

What does the operator >= represent?

A. Strictly greater than
B. Greater than or equal to
C. Equal to
D. Not equal to

Ans: B

Explanation: >= is the 'greater than or equal to' relational operator, true when the left operand is either greater than or exactly equal to the right operand.

Q100 Relational Operators *Meaning*

What does the operator <= represent?

A. Less than
B. Less than or equal to
C. Greater than
D. Equal to

Ans: B

Explanation: <= is the 'less than or equal to' relational operator, true when the left operand is either less than or exactly equal to the right operand.

Q101 Relational Operators *Purpose*

Relational operators in C always produce a result of data type:

A. float
B. int (0 or 1)
C. char
D. double

Ans: B

Explanation: In C, relational expressions evaluate to an int: 1 for true and 0 for false.

Q102 Relational Operators *Usage*

What is the output of: printf("%d", 7 == 7.0);

A. 0
B. 1
C. 7
D. Compilation error

Ans: B

Explanation: C performs implicit type conversion; 7 (int) is promoted for comparison with 7.0 (double), and since the values are numerically equal, the result is 1 (true).

Q103 Relational Operators *Evaluation of Conditions*

What is the result of the expression: 10 != 10?

A. 0 **B.** 1

C. 10 **D.** -1

Ans: A

Explanation: Since 10 is indeed equal to 10, the 'not equal to' test is false, giving a result of 0.

Q104 Relational Operators *Code-Based*

What is the output of the following code?

```
int a = 5, b = 8;
printf("%d", a < b);
```

A. 0 **B.** 1

C. 5 **D.** 8

Ans: B

Explanation: a(5) is indeed less than b(8), so the relational expression a < b evaluates to 1 (true).

Q105 Relational Operators *Output-Based*

What is the output of the following code?

```
printf("%d %d", 5 > 3, 3 > 5);
```

A. 1 1 **B.** 0 0

C. 1 0 **D.** 0 1

Ans: C

Explanation: 5 > 3 is true (1), and 3 > 5 is false (0), so the output is '1 0'.

Q106 Relational Operators *Tricky*

What is a common bug caused by confusing = and == in a condition?

A. It always causes a compilation error, so it is easy to catch

B. The assignment (=) silently assigns a value and the condition becomes based on the assigned value's truthiness, often causing unintended logic

C. It has no effect on program logic

D. C automatically converts = to == inside if conditions

Ans: B

Explanation: Since if (a = 5) is legal C syntax, it silently assigns 5 to a and the condition becomes true (as 5 is non-zero), which can introduce subtle logic bugs without a compiler error.

Q107 Relational Operators *Applications*

Relational operators are essential in decision structures because they:

A. Perform looping automatically

B. Generate the true/false conditions that if, while, and switch structures depend on

C. Allocate dynamic memory

D. Convert data types automatically

Ans: B

Explanation: Decision structures like if and loops like while rely on relational (and logical) operators to generate the Boolean condition that controls their behavior.

Q108 Relational Operators *Comparison-Based*

How do relational operators differ from logical operators?

A. Relational operators compare two values; logical operators combine multiple Boolean conditions

B. They are identical in function

C. Logical operators can only be used with strings

D. Relational operators cannot be used inside if statements

Ans: A

Explanation: Relational operators (>, <, ==, etc.) compare individual values, while logical operators (&&, ||, !) combine or invert the results of such comparisons.

Q109 Relational Operators *Difficult*

What is the output of: printf("%d", (5 > 3) == 1);

A. 0

B. 1

C. 5

D. 3

Ans: B

Explanation: (5 > 3) evaluates to 1; then 1 == 1 is compared, which is true, so the final result printed is 1.

Section 8 of 13

Conditional Expression (Ternary) Operator

Q110 Conditional Expression (Ternary) Operator *Definition-Based*

Which operator is known as the conditional operator in C?

A. &&
B. ||
C. ?:
D. ==

Ans: C

Explanation: The ?: operator is called the conditional or ternary operator because it evaluates a condition and returns one of two expressions based on the result.

Q111 Conditional Expression (Ternary) Operator *Syntax-Based*

What is the correct syntax for the ternary operator?

A. condition ? expr1 : expr2
B. condition : expr1 ? expr2
C. if (condition) expr1 : expr2
D. condition ?? expr1 :: expr2

Ans: A

Explanation: The correct syntax is condition ? expression_if_true : expression_if_false — a single expression that evaluates to one of two possible values.

Q112 Conditional Expression (Ternary) Operator *Working*

How does the ternary operator evaluate an expression?

A. It always evaluates both expr1 and expr2
B. It evaluates the condition first; if true, it returns expr1, otherwise it returns expr2 (only one of them is evaluated)
C. It evaluates expr2 first, then the condition
D. It ignores the condition and always returns expr1

Ans: B

Explanation: The ternary operator evaluates the condition, then evaluates and returns only the corresponding expression (expr1 for true, expr2 for false) — the other expression is never evaluated.

Q113 Conditional Expression (Ternary) Operator *Why 'Ternary'*

Why is ?: called a 'ternary' operator?

A. Because it can only be used three times in a program

B. Because it operates on three operands: a condition and two expressions

C. Because it returns three possible values

D. Because it requires three semicolons

Ans: B

Explanation: It is called ternary because it takes exactly three operands: the condition, the true-expression, and the false-expression.

Q114 Conditional Expression (Ternary) Operator *Code-Based*

What is the output of the following code?

```
int a = 10, b = 20, max;
max = (a > b) ? a : b;
printf("%d", max);
```

A. 10

B. 20

C. 30

D. 0

Ans: B

Explanation: a > b (10 > 20) is false, so the ternary expression evaluates to b (20), which is assigned to max and printed.

Q115 Conditional Expression (Ternary) Operator *Output-Based*

What is the output of the following code?

```
int n = 7;
printf(n % 2 == 0 ? "Even" : "Odd");
```

A. Even

B. Odd

C. 7

D. 0

Ans: B

Explanation: 7 % 2 equals 1, which is not equal to 0, so the condition is false, causing the ternary to select and print 'Odd'.

Q116 **Conditional Expression (Ternary) Operator** *Advantages*

What is a key advantage of the ternary operator over an equivalent if-else?

A. It can perform loops

B. It is a compact expression that can be used inline (e.g., directly inside an assignment or printf), unlike if-else statements

C. It removes the need for relational operators

D. It always executes faster on all compilers

Ans: B

Explanation: Because ?: is an expression (not a statement), it can be embedded directly within assignments, function calls, or other expressions — something if-else cannot do directly.

Q117 **Conditional Expression (Ternary) Operator** *Comparison with if-else*

Which statement correctly compares the ternary operator with if-else?

A. Ternary can only be used for arithmetic operations, if-else cannot

B. Ternary is best for simple, single-expression decisions, while if-else is better for executing multiple statements or complex logic

C. if-else is an expression while ternary is a statement

D. They cannot be used interchangeably in any scenario

Ans: B

Explanation: The ternary operator suits simple, single-value decisions embedded in expressions, whereas if-else is more appropriate when multiple statements or complex logic blocks are needed.

Q118 **Conditional Expression (Ternary) Operator** *Applications*

Which scenario is most appropriate for using the ternary operator?

A. Executing five different statements based on a condition

B. Assigning the larger of two values to a variable in a single line

C. Looping through an array

D. Declaring a structure

Ans: B

Explanation: The ternary operator is ideal for simple, single-value decisions such as picking the maximum of two values in one concise line.

Q119 Conditional Expression (Ternary) Operator *Fill-in-the-Blank*

In the ternary expression condition ? expr1 : expr2, if condition evaluates to false (0), the result of the expression is _____.

A. expr1
B. expr2
C. condition
D. undefined

Ans: B

Explanation: When the condition is false, the ternary operator evaluates and returns expr2 (the part after the colon).

Q120 Conditional Expression (Ternary) Operator *Tricky*

What is the output of: printf("%d", 1 ? 2 ? 3 : 4 : 5); (nested ternary)

A. 2
B. 3
C. 4
D. 5

Ans: B

Explanation: Ternary operators are right-associative. The outer condition (1) is true, so it evaluates '2 ? 3 : 4', where 2 is true, resulting in 3.

Q121 Conditional Expression (Ternary) Operator *Statement-Based*

Which of the following is TRUE regarding the ternary operator?

A. It can replace any if-else, including those with multiple statements per branch
B. It is an expression that evaluates to a single value, best suited for simple two-way value selection
C. It cannot be nested
D. It requires braces { } around each expression

Ans: B

Explanation: The ternary operator produces a single resulting value and is not designed to execute multiple statements per branch the way if-else blocks can.

Q122 Conditional Expression (Ternary) Operator *Assertion-Reason*

Assertion (A): The expression (a > b) ? a : b can be used to find the maximum of two numbers.
Reason (R): The ternary operator returns the first expression if the condition is true, otherwise the second expression.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. Both A and R are true, but R does not explain A
C. A is true, R is false
D. A is false, R is true

Ans: A

Explanation: Since the ternary returns a when a>b is true (a is the larger value) and b otherwise, R correctly explains why this expression finds the maximum, validating A.

Q123 Conditional Expression (Ternary) Operator *Difficult*

What is the output of the following code?

```
int x = 5;
int y = (x > 0) ? (x > 10 ? 1 : 2) : 3;
printf("%d", y);
```

A. 1
B. 2
C. 3
D. 5

Ans: B

Explanation: x>0 (5>0) is true, so the nested ternary (x>10 ? 1 : 2) is evaluated. x>10 (5>10) is false, so it returns 2, which is assigned to y.

Q124 Conditional Expression (Ternary) Operator *Previous-Year Pattern*

Which precedence/associativity rule applies to the ternary operator in C?

A. It is left-associative
B. It is right-associative
C. It has no defined associativity
D. It always evaluates left-to-right regardless of nesting

Ans: B

Explanation: The C ternary operator is right-associative, meaning in nested ternary expressions, grouping proceeds from right to left.

Section 9 of 13

The Switch Statement

Q125 The Switch Statement *Definition-Based*

What is the primary purpose of the switch statement in C?

A. To repeat a block of code

B. To select one among many code blocks to execute based on the value of an expression

C. To declare multiple variables

D. To perform pointer arithmetic

Ans: B

Explanation: The switch statement provides a multi-way branching mechanism, selecting and executing the block matching the value of a single controlling expression.

Q126 The Switch Statement *Syntax-Based*

Which is the correct general syntax for a switch statement?

A. switch(expr) { case val1: stmt; case val2: stmt; }

B. switch expr { case val1: stmt; }

C. switch(expr): case val1 stmt;

D. switch { expr case val1: stmt; }

Ans: A

Explanation: The correct syntax encloses the switch expression in parentheses, followed by a braces block containing one or more case labels: switch(expr) { case val1: stmt; ... }

Q127 The Switch Statement *Execution Mechanism*

How does the switch statement decide which case to execute?

A. It always executes the first case

B. It compares the switch expression's value with each case label's constant value and transfers control to the matching case

C. It executes all cases sequentially

D. It randomly selects a case

Ans: B

Explanation: The switch expression is evaluated once, and control jumps to the case label whose constant matches that value; if no match is found, control goes to the default label (if present).

Q128 The Switch Statement *Control Transfer*

Which data types can the switch expression validly evaluate to in standard C?

A. float or double

B. Integer or character type (integral types)

C. String (char array)

D. Structure type

Ans: B

Explanation: The switch expression in C must evaluate to an integral type (int or char); floating-point types, strings, and structures are not permitted directly.

Q129 The Switch Statement *Applications*

The switch statement is most suitable when:

A. Testing complex range-based conditions

B. Selecting among several discrete, known constant values of a single variable

C. Testing conditions involving multiple different variables simultaneously

D. Performing floating-point comparisons

Ans: B

Explanation: switch excels when a single expression must be compared against a fixed set of discrete constant values, such as menu choices or grade letters.

Q130 The Switch Statement *Restrictions*

Which of the following is a restriction of the switch statement in C?

A. Case labels must be constant expressions known at compile time

B. Case labels can be variables

C. switch can test floating-point expressions directly

D. There is no limit and any expression type is allowed as case label

Ans: A

Explanation: In C, each case label must be a constant expression (evaluable at compile time); using a variable as a case label causes a compilation error.

Q131 The Switch Statement *Code-Based*

What is the output of the following code?

```
int day = 3;
switch(day) {
case 1: printf("Mon"); break;
case 2: printf("Tue"); break;
case 3: printf("Wed"); break;
default: printf("Invalid");
}
```

A. Mon
B. Tue
C. Wed
D. Invalid

Ans: C

Explanation: day equals 3, which matches case 3, so 'Wed' is printed, and the break exits the switch.

Q132 The Switch Statement *Output-Based*

What is the output of the following code?

```
int x = 10;
switch(x) {
case 1: printf("One"); break;
case 2: printf("Two"); break;
default: printf("Other");
}
```

A. One
B. Two
C. Other
D. No output

Ans: C

Explanation: Since x(10) does not match any listed case (1 or 2), control transfers to the default label, printing 'Other'.

Q133 The Switch Statement *Flow Analysis*

What happens if a switch statement has no default label and the expression matches no case?

A. A compilation error occurs

B. The program crashes

C. No block executes; control simply exits the switch statement

D. The first case executes automatically

Ans: C

Explanation: If no case matches and there is no default label, none of the case blocks execute; the program simply continues after the closing brace of the switch.

Q134 The Switch Statement *Scenario-Based*

A menu-driven calculator program needs to perform +, -, *, / based on user's single-character choice. Which construct is most appropriate?

A. A single relational operator

B. switch statement

C. Ternary operator only

D. A compound if without else

Ans: B

Explanation: Since the choice is a discrete character value among a known fixed set (+, -, *, /), switch is the cleanest and most readable construct.

Q135 The Switch Statement *Comparison-Based*

How does switch differ from an else-if ladder in terms of the type of condition it can test?

A. switch can test range conditions like x > 10, else-if cannot

B. switch tests equality of a single expression against constants; else-if can test any Boolean expression including ranges and multiple variables

C. They are functionally identical in every respect

D. switch supports floating-point comparisons better than else-if

Ans: B

Explanation: switch is restricted to equality matching of one expression against constant values, while else-if ladders can evaluate arbitrary Boolean expressions, including ranges and multiple variables.

Q136 The Switch Statement *Match-the-Following*

Match the switch component with its role:
(1) switch expression (2) case label (3) default
(a) fallback when no match found (b) value being compared (c) constant to match against

A. 1-b, 2-c, 3-a
B. 1-a, 2-b, 3-c
C. 1-c, 2-a, 3-b
D. 1-b, 2-a, 3-c

Ans: A

Explanation: The switch expression is the value being compared (b), case labels are the constants to match against (c), and default is the fallback when no case matches (a).

Q137 The Switch Statement *Tricky*

What is the output of the following code (note the duplicate case values are NOT present, but check case order)?

```
int grade = 2;
switch(grade) {
default: printf("Unknown"); break;
case 1: printf("First"); break;
case 2: printf("Second"); break;
}
```

A. Unknown
B. First
C. Second
D. Compilation error

Ans: C

Explanation: Even though default appears first in the code, switch still matches grade(2) against case 2 correctly and executes 'Second' — case order in source code does not affect matching.

Q138 The Switch Statement *Fill-in-the-Blank*

Each case label's constant value in a switch statement must be _____ within the same switch block.

A. duplicated at least once
B. unique
C. in ascending order
D. a floating-point number

Ans: B

Explanation: Duplicate case values within the same switch statement are illegal and cause a compilation error; each case value must be unique.

Q139 The Switch Statement *Assertion-Reason*

Assertion (A): switch is often more efficient than a long else-if ladder for many discrete integer comparisons.
Reason (R): Some compilers can optimize switch statements using jump tables instead of sequential comparisons.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: Jump-table optimization (when applicable) allows the compiler to jump directly to the matching case in constant time, rather than testing conditions sequentially as an else-if ladder does, which correctly explains the efficiency claim.

Q140 The Switch Statement *Difficult*

What is the output of the following code, where the default label is placed in the middle?

```
int x = 5;
switch(x) {
case 1: printf("A"); break;
default: printf("D"); break;
case 5: printf("E"); break;
}
```

A. A

B. D

C. E

D. DE

Ans: C

Explanation: x(5) matches case 5 exactly, so control jumps directly to case 5 and prints 'E', regardless of where default appears in the block.

Q141 The Switch Statement *Previous-Year Pattern*

Which of the following expressions is INVALID as a switch expression in standard C?

A. switch(x) where x is int

B. switch(ch) where ch is char

C. switch(x + y) where x, y are int

D. switch(3.14) using a float literal

Ans: D

Explanation: The switch expression must evaluate to an integral type; using a float literal directly is invalid in standard C and causes a compilation error.

Section 10 of 13

Switch Statement with break

Q142 Switch Statement with break *Definition-Based*

What is the purpose of the break statement inside a switch block?

A. To start a new loop

B. To terminate the switch statement immediately, preventing fall-through into subsequent cases

C. To skip to the default label always

D. To declare a new case label

Ans: B

Explanation: The break statement terminates a switch block immediately and prevents execution from continuing into subsequent case labels.

Q143 Switch Statement with break *Purpose of break*

Why is break commonly placed at the end of each case block?

A. To make the code longer

B. To ensure only the matched case's code executes, avoiding unintended execution of following cases

C. It is required by C syntax and the program won't compile without it

D. To improve variable declaration

Ans: B

Explanation: Without break, execution 'falls through' into the next case regardless of whether it matches, so break is used to confine execution strictly to the matched case.

Q144 Switch Statement with break *Execution Behavior*

What is the output of the following code?

```
int x = 2;
switch(x) {
case 1: printf("One"); break;
case 2: printf("Two"); break;
case 3: printf("Three"); break;
}
```

A. One
B. Two
C. Three
D. OneTwoThree

Ans: B

Explanation: x matches case 2, printing 'Two'; the break statement then exits the switch immediately, so no other cases execute.

Q145 Switch Statement with break *Output-Based*

What is the output of the following code?

```
char grade = 'B';
switch(grade) {
case 'A': printf("Excellent"); break;
case 'B': printf("Good"); break;
case 'C': printf("Average"); break;
default: printf("Invalid");
}
```

A. Excellent
B. Good
C. Average
D. Invalid

Ans: B

Explanation: grade matches case 'B', so 'Good' is printed; break then exits the switch, skipping the remaining cases and default.

Q146 Switch Statement with break *Advantages*

What is a key advantage of always using break at the end of each case in a menu-driven program?

A. It makes the program run in a loop automatically

B. It ensures clean, predictable execution of exactly one case, avoiding unintended fall-through bugs

C. It removes the need for a default label

D. It converts the switch into an if-else automatically

Ans: B

Explanation: Using break in every case guarantees that only the intended matching block executes, making program behavior predictable and free from accidental fall-through errors.

Q147 Switch Statement with break *Code-Based*

What is the output of the following code?

```
int choice = 4;
switch(choice) {
case 1: printf("A"); break;
case 2: printf("B"); break;
default: printf("Default"); break;
}
```

A. A

B. B

C. Default

D. No output

Ans: C

Explanation: choice(4) matches neither case 1 nor case 2, so control goes to default, printing 'Default'; the break there is good practice even though default is typically last.

Q148 Switch Statement with break *Scenario-Based*

In a traffic light simulation, each color (Red, Yellow, Green) must trigger only its own specific action with no overlap. Which switch design is correct?

A. Switch without any break statements

B. Switch with a break statement at the end of every case, including default

C. Switch with break only in the first case

D. Avoid switch entirely and use nested loops

Ans: B

Explanation: Since each color must trigger only its own action without spilling into the next case, break must be used consistently after every case block.

Q149 Switch Statement with break *Applications*

Which scenario benefits most from switch-with-break rather than switch-without-break?

A. Grouping multiple related cases to share one action

B. Ensuring each discrete option executes independently and exclusively, such as a menu selection

C. Counting the number of matching cases

D. Executing all cases regardless of the input

Ans: B

Explanation: switch-with-break is ideal when each case must run in isolation, such as when a user selects one specific menu option among several.

Q150 Switch Statement with break *Tricky*

What is the output of the following code if the break statement in case 2 is accidentally omitted?

```
int x = 2;
switch(x) {
case 1: printf("A");
case 2: printf("B");
case 3: printf("C"); break;
case 4: printf("D");
}
```

A. B

B. BC

C. ABC

D. BCD

Ans: B

Explanation: x matches case 2 and prints 'B'; since there's no break after case 2, execution falls through into case 3, printing 'C' as well, before the break in case 3 stops execution — output is 'BC'.

Q151 Switch Statement with break *Fill-in-the-Blank*

The break statement, when executed inside a switch, transfers control to the _____.

A. beginning of the switch block

B. statement immediately following the switch block

C. default label only

D. next case label

Ans: B

Explanation: break immediately exits the entire switch construct, and execution resumes at the first statement following the closing brace of the switch block.

Q152 Switch Statement with break *Comparison-Based*

How does the presence of break in every case affect the behavior compared to a switch with no break statements at all?

A. No difference; both always execute all cases

B. With break, only the matched case executes; without break, execution falls through and continues into subsequent cases until a break or the end of switch is reached

C. Break statements are ignored by the compiler

D. Switch without break causes a compilation error

Ans: B

Explanation: break confines execution to just the matched case, while omitting it causes fall-through, where all subsequent case bodies execute sequentially until a break is encountered or the switch ends.

Q153 Switch Statement with break *Difficult*

What is the output of the following code?

```
int n = 1;
switch(n) {
case 1:
printf("1");
break;
case 2:
printf("2");
break;
case 1:
printf("1 again");
break;
}
```

A. 1

B. 1 again

C. 11 again

D. Compilation error due to duplicate case value

Ans: D

Explanation: Duplicate case labels (case 1 appearing twice) are not allowed in the same switch block in C, and this results in a compilation error, regardless of the break statements.

Q154 Switch Statement with break *Previous-Year Pattern*

Which of the following best describes best practice regarding the default case and break in exam-style switch programs?

A. default should never include a break statement

B. Placing break after default is good practice, especially if default is not the last label, to prevent unintended fall-through

C. default cannot be used with break at all

D. break has no effect when placed after default

Ans: B

Explanation: Even though default is often placed last, if it appears earlier in the switch block, including a break after it prevents accidental fall-through into subsequent case labels.

Q155 Switch Statement with break *Assertion-Reason*

Assertion (A): break is optional in the last case of a switch statement.
Reason (R): There are no further case labels for control to fall through into after the last case.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: Since the last case has no subsequent case to fall through into, omitting break there has no functional difference — R correctly explains why A is true.

Section 11 of 13

Switch Statement without break

Q156 **Switch Statement without break** *Definition-Based*

What is 'fall-through' behavior in a switch statement?

A. The switch statement fails to compile

B. Execution continues into subsequent case blocks after a matched case, because no break statement stops it

C. The switch always jumps to default

D. The program crashes at runtime

Ans: B

Explanation: Fall-through occurs when a matched case block has no break statement, causing execution to continue sequentially into the code of the next case label(s), regardless of whether they match.

Q157 **Switch Statement without break** *Fall-through Concept*

In a switch without break, once a matching case is found, when does execution finally stop?

A. Immediately after the matched case's first statement

B. It continues executing all remaining case bodies (and default) until the end of the switch block, unless a break is encountered along the way

C. It never executes anything

D. Only the default block executes

Ans: B

Explanation: Without break statements, execution 'falls through' every subsequent case body in sequence until the switch block ends or a break is encountered.

Q158 Switch Statement without break *Execution Sequence*

What is the output of the following code (no break statements)?

```
int x = 1;
switch(x) {
case 1: printf("A");
case 2: printf("B");
case 3: printf("C");
}
```

A. A
B. AB
C. ABC
D. No output

Ans: C

Explanation: x matches case 1, printing 'A'; with no break, execution falls through case 2 (prints 'B') and case 3 (prints 'C'), giving 'ABC'.

Q159 Switch Statement without break *Output-Based*

What is the output of the following code?

```
int x = 2;
switch(x) {
case 1: printf("One");
case 2: printf("Two");
case 3: printf("Three");
default: printf("Def");
}
```

A. Two
B. TwoThree
C. TwoThreeDef
D. Def

Ans: C

Explanation: x matches case 2, printing 'Two'; without break, execution falls through case 3 ('Three') and then default ('Def'), producing 'TwoThreeDef'.

Q160 Switch Statement without break *Applications*

Fall-through (switch without break) is intentionally useful when:

A. Each case must behave completely independently

B. Multiple case values should share and execute the same block of code

C. Only exact numeric matches are desired with no shared behavior

D. The switch expression is a floating-point number

Ans: B

Explanation: Deliberate fall-through is useful for grouping multiple case labels that should trigger identical behavior, such as treating both 'a' and 'A' the same way.

Q161 Switch Statement without break *Scenario-Based*

A program needs to treat weekend days (Saturday and Sunday) with the same message 'Weekend', while weekdays get individual messages. Which switch design fits best?

A. switch with break after every single case, including Saturday

B. switch where case 'Saturday' has no break, deliberately falling through into case 'Sunday' to share the same 'Weekend' output

C. Avoid switch entirely

D. Use only if-else with no relational operators

Ans: B

Explanation: Grouping case 'Saturday' and case 'Sunday' together (with fall-through from Saturday into Sunday) allows both to execute the same shared code block for 'Weekend'.

Q162 Switch Statement without break *Code-Based*

What is the output of the following intentional fall-through code?

```
char ch = 'a';
switch(ch) {
case 'a':
case 'e':
case 'i':
case 'o':
case 'u':
printf("Vowel");
break;
default:
printf("Consonant");
}
```

A. Vowel
B. Consonant
C. a
D. No output

Ans: A

Explanation: Since case 'a' has no code before falling through to the shared printf under case 'u', matching 'a' still reaches and executes printf("Vowel"), followed by break.

Q163 Switch Statement without break *Tricky*

What is the output of the following code?

```
int x = 5;
switch(x) {
case 1: printf("A"); break;
case 5: printf("E");
case 6: printf("F"); break;
case 7: printf("G");
}
```

A. E
B. EF
C. EFG
D. F

Ans: B

Explanation: x matches case 5, printing 'E'; with no break, it falls into case 6, printing 'F'; the break in case 6 then stops execution before reaching case 7 — output is 'EF'.

Q164 Switch Statement without break *Comparison-Based*

What is the primary risk of switch statements without deliberate break placement?

A. The program will not compile

B. Unintended fall-through can cause multiple unrelated case blocks to execute accidentally, producing incorrect output

C. It always improves performance with no downsides

D. It disables the default label

Ans: B

Explanation: If break is accidentally omitted where it's needed, unrelated subsequent cases execute unintentionally, which is a common source of logic bugs in switch statements.

Q165 Switch Statement without break *Fill-in-the-Blank*

When a switch statement lacks break statements, execution continues into subsequent cases; this behavior is known as _____.

A. short-circuiting

B. fall-through

C. nesting

D. casting

Ans: B

Explanation: The term for continuing execution into subsequent case labels without an intervening break is 'fall-through'.

Q166 Switch Statement without break *Assertion-Reason*

Assertion (A): Fall-through behavior can be used intentionally as a programming technique.
Reason (R): Grouping multiple case labels together without break lets them share the same block of executable code.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: Deliberately omitting break to let multiple case labels fall through into shared code is a legitimate and common technique, and R correctly explains why this is useful.

Q167 Switch Statement without break *Difficult*

What is the output of the following code?

```
int x = 3;
switch(x) {
default: printf("D");
case 1: printf("1");
case 2: printf("2");
case 3: printf("3");
}
```

A. D

B. 3

C. D123

D. 123

Ans: B

Explanation: x matches case 3 directly (case matching does not depend on order), so execution starts at case 3, printing '3'; there are no further cases after it and no break needed, so output is simply '3'.

Q168 Switch Statement without break *Previous-Year Pattern*

Which statement about switch without break and 'default' interaction is TRUE?

A. default always executes first regardless of match

B. If default lacks a break and is not the last label, execution can fall through into subsequent case blocks even if their values don't match the switch expression

C. default can never fall through

D. default is mandatory in every switch statement

Ans: B

Explanation: The default label behaves like any other label with respect to fall-through: if it lacks a break and isn't the last block, execution proceeds into whatever code follows it, irrespective of case value matching.

Section 12 of 13

Concept of Case Label

Q169 Concept of Case Label *Definition-Based*

What is a case label in a switch statement?

A. A variable that stores the switch expression's value

B. A constant value against which the switch expression is compared, marking the start of a code block

C. The default block of the switch

D. A loop control variable

Ans: B

Explanation: A case label is a constant value (like case 5: or case 'A':) that marks a specific branch of code to execute when the switch expression matches that constant.

Q170 Concept of Case Label *Rules*

Which of the following is a mandatory rule for case labels in C?

A. Case labels must be variables

B. Case labels must be constant expressions evaluable at compile time

C. Case labels must be floating-point values

D. Case labels can be duplicated freely

Ans: B

Explanation: Every case label must be a compile-time constant expression; using a runtime variable as a case label results in a compilation error.

Q171 Concept of Case Label *Syntax*

Which is the correct syntax for writing a case label?

A. case value; statement;

B. case value: statement;

C. case(value) statement;

D. case value -> statement;

Ans: B

Explanation: A case label in C is written as case constant_value: followed by the statements to execute, using a colon rather than a semicolon or arrow.

Q172 Concept of Case Label *Characteristics*

Which of the following is a valid case label in C?

A. case x: where x is a variable

B. case 5+3: (a constant expression)

C. case 3.14: (a floating-point literal)

D. case "hello": (a string literal)

Ans: B

Explanation: 5+3 is a valid constant expression (evaluates to 8 at compile time). Variables, floating-point literals, and strings are not valid case labels in standard C switch statements.

Q173 Concept of Case Label *Valid Case Labels*

Which of the following data types can be validly used for case label constants?

A. int and char constants

B. float and double constants

C. String literals

D. Structure constants

Ans: A

Explanation: Case labels must be integral constants — either int or char literals (char is promoted to its integer ASCII value); floating-point and string types are not allowed.

Q174 Concept of Case Label *Constant Expressions*

Which of the following is TRUE about constant expressions used as case labels?

A. They can include function calls that return a value at runtime

B. They must be fully evaluable by the compiler at compile time, such as literals or expressions of only constants

C. They must always be written as a single literal digit

D. They can include user input variables

Ans: B

Explanation: Constant expressions for case labels must be resolvable entirely at compile time (e.g., literals, enum constants, or arithmetic on constants) — no runtime values like function calls or variables are permitted.

Q175 Concept of Case Label *Usage within Switch*

What happens if two case labels in the same switch statement have the same constant value?

A. The compiler picks the first one and ignores the second silently

B. It results in a compilation error due to duplicate case value

C. Both cases execute together every time

D. The program runs but produces unpredictable output

Ans: B

Explanation: Duplicate case values within the same switch statement are illegal in C and are caught as a compilation error, since each value must uniquely identify one branch.

Q176 Concept of Case Label *Code-Based*

Which of the following switch blocks contains an invalid case label and will fail to compile?
Option analysis assumes: int a = 5; (variable, not constant)

A. case 10:

B. case 'A':

C. case a: (where a is a variable)

D. case 2+3:

Ans: C

Explanation: case a: is invalid because a is a variable, not a compile-time constant; C requires all case labels to be fixed constant expressions.

Q177 Concept of Case Label *Output-Based*

What is the output of the following code using a character case label?

```
char grade = 'B';
switch(grade) {
case 'A': printf("90-100"); break;
case 'B': printf("80-89"); break;
case 'C': printf("70-79"); break;
}
```

A. 90-100

B. 80-89

C. 70-79

D. No output

Ans: B

Explanation: grade matches the case label 'B' (compared via its ASCII integer value), so '80-89' is printed.

Q178 Concept of Case Label *Scenario-Based*

A programmer wants to use case (marks/10): as a case label, where marks is a variable. Is this valid?

A. Yes, any expression is valid as a case label

B. No, because marks/10 depends on a runtime variable and is not a compile-time constant

C. Yes, but only if marks is declared as const

D. Yes, if marks/10 always evaluates to an integer

Ans: B

Explanation: Even simple-looking expressions like marks/10 are invalid case labels if marks is a regular runtime variable, since the value cannot be determined at compile time.

Q179 Concept of Case Label *Comparison-Based*

How does a case label differ from an if condition in terms of what it can express?

A. A case label can express any Boolean condition, just like if

B. A case label can only express exact equality against a single compile-time constant, while an if condition can express any Boolean expression including ranges

C. They are functionally identical

D. case labels can use relational operators like > and <

Ans: B

Explanation: Case labels are restricted to exact-match constants, whereas if conditions can use relational and logical operators to express far more complex and flexible conditions, including ranges.

Q180 Concept of Case Label *Tricky*

Which of the following is technically a valid case label, even though it might seem unusual to beginners?

A. case 5*2-3: (an arithmetic constant expression)

B. case x+1: where x is a non-const variable

C. case getValue(): where getValue() is a function call

D. case "5": (a string)

Ans: A

Explanation: 5*2-3 is a valid constant expression because it involves only literal constants and evaluates to 7 entirely at compile time, unlike the other options which depend on runtime values or are the wrong type.

Q181 Concept of Case Label *Fill-in-the-Blank*

The value used in a case label must be a(n) _____ that the compiler can resolve before the program runs.

A. runtime variable
B. integer or character constant expression
C. floating-point variable
D. string variable

Ans: B

Explanation: Case labels require constant expressions of integral type (int or char) that are fully resolved at compile time, not runtime variables or floating-point/string values.

Q182 Concept of Case Label *Assertion-Reason*

Assertion (A): case 'A': and case 65: behave identically in a switch statement testing a char expression.
Reason (R): Character constants in C are internally represented and compared using their ASCII integer values.
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. Both A and R are true, but R does not explain A
C. A is true, R is false
D. A is false, R is true

Ans: A

Explanation: Since 'A' has the ASCII value 65, both case labels represent the exact same constant internally, and R correctly explains why they behave identically.

Q183 Concept of Case Label *Difficult*

What is the output of the following code that mixes character and equivalent ASCII integer case labels?

```
char ch = 66;
switch(ch) {
case 'A': printf("Apple"); break;
case 66: printf("Ball"); break;
case 'C': printf("Cat"); break;
}
```

A. Apple
B. Ball
C. Cat
D. Compilation error

Ans: B

Explanation: ch has the value 66, which corresponds exactly to ASCII 'B'. Since case 66: matches ch's value directly, 'Ball' is printed.

Q184 **Concept of Case Label** *Previous-Year Pattern*

Which of the following case label sets would cause a compilation error in the same switch block?

A. case 1: and case 2:

B. case 'A': and case 66: (where 'A' has ASCII value 65)

C. case 1: and case 1: (identical values)

D. case 10: and case 20:

Ans: C

Explanation: Having two case labels with the exact same constant value (case 1: appearing twice) is illegal and causes a duplicate case label compilation error.

Section 13 of 13

Mixed Review: Cross-Topic MCQs

Q185 Mixed Review: Cross-Topic MCQs *Comparison-Based*

Which construct would you choose to test 5 mutually exclusive numeric range conditions (e.g., grade bands), and why?

A. switch, because ranges map directly to case labels

B. else-if ladder, because it can evaluate relational range conditions that switch cannot express directly

C. A single if statement without else

D. The ternary operator, because it supports unlimited branches

Ans: B

Explanation: switch tests exact equality against constants, not ranges, so an else-if ladder (using relational operators) is the correct tool for range-based multi-way decisions.

Q186 Mixed Review: Cross-Topic MCQs *Comparison-Based*

Which construct would you choose to select one of 7 fixed menu options entered as an integer 1–7?

A. switch statement

B. Ternary operator only

C. A single relational operator

D. Logical NOT operator

Ans: A

Explanation: Since the input is a discrete set of known integer values, switch offers the cleanest, most readable multi-way branching for menu-driven programs.

Q187 Mixed Review: Cross-Topic MCQs *Match-the-Following*

Match each construct to its best-fit use case:
(1) if (2) if-else (3) else-if ladder (4) switch
(a) Menu selection by exact value (b) Single condition, no alternative needed (c) Multiple ranked range conditions (d) Binary (two-outcome) decision

A. 1-b, 2-d, 3-c, 4-a

B. 1-a, 2-b, 3-c, 4-d

C. 1-c, 2-a, 3-b, 4-d

D. 1-d, 2-c, 3-a, 4-b

Ans: A

Explanation: if fits single-condition checks (b), if-else fits binary decisions (d), else-if ladder fits ranked range conditions (c), and switch fits exact-value menu selection (a).

Q188 Mixed Review: Cross-Topic MCQs *Application-Oriented*

A billing system applies different tax slabs based on income ranges. Which is the most appropriate control structure?

A. switch statement using income as the expression

B. else-if ladder using relational operators to test income ranges

C. A single if statement

D. The NOT (!) operator alone

Ans: B

Explanation: Since income slabs are ranges rather than discrete exact values, an else-if ladder with relational operators is the correct approach; switch cannot directly test ranges.

Q189 Mixed Review: Cross-Topic MCQs *Application-Oriented*

To assign the smaller of two numbers to a variable in a single, compact line of code, which is most suitable?

A. Nested if statement

B. switch without break

C. Ternary (conditional) operator

D. else-if ladder

Ans: C

Explanation: The ternary operator's compact single-expression nature makes it ideal for simple value-selection tasks like picking the smaller of two numbers in one line.

Q190 Mixed Review: Cross-Topic MCQs *Comparison-Based*

Which two constructs both allow only ONE of several blocks to execute for a given run, ensuring mutual exclusivity?

A. if-else and else-if ladder

B. Nested if and logical AND only

C. Compound relational test and case label

D. Relational operator and ternary operator only

Ans: A

Explanation: Both if-else and the else-if ladder are designed so that exactly one branch executes per run — if-else offers two mutually exclusive paths, and else-if ladder extends this to many.

Q191 Mixed Review: Cross-Topic MCQs *Assertion-Reason*

Assertion (A): switch statements are generally not suitable for testing floating-point conditions.
Reason (R): The switch expression and case labels in C must be of integral type (int or char).
Choose the correct option:

A. Both A and R are true, and R correctly explains A
B. Both A and R are true, but R does not explain A
C. A is true, R is false
D. A is false, R is true

Ans: A

Explanation: Because C restricts switch expressions and case labels to integral types, floating-point conditions cannot be tested directly with switch, which is exactly why R explains A.

Q192 Mixed Review: Cross-Topic MCQs *Tricky*

Which of the following correctly ranks these constructs from MOST restrictive matching (exact only) to LEAST restrictive (any Boolean expression)?

A. switch → else-if ladder → simple if
B. simple if → switch → else-if ladder
C. else-if ladder → simple if → switch
D. switch → simple if → else-if ladder

Ans: A

Explanation: switch only allows exact constant matching (most restrictive); simple if evaluates one Boolean expression; else-if ladder evaluates a whole chain of independent Boolean expressions (least restrictive/most flexible).

Q193 Mixed Review: Cross-Topic MCQs *Fill-in-the-Blank*

A construct that combines the compactness of a single expression with two-way decision-making, often used inside printf or assignments, is called the _____ operator.

A. relational
B. logical
C. ternary (conditional)
D. switch

Ans: C

Explanation: The ternary (?:) operator uniquely combines decision logic into a single expression usable inline, unlike statement-based constructs like if-else or switch.

Q194 Mixed Review: Cross-Topic MCQs *Scenario-Based*

A vending machine accepts a coin value (1, 2, 5, or 10) and must dispense a specific item for each exact value, with an error message for any other coin. Best construct?

A. switch statement with a default case for invalid coins

B. Ternary operator chained multiple times

C. A single if without else

D. Compound relational test using only OR

Ans: A

Explanation: Since the input is a small set of exact discrete values with a common 'invalid' case, switch with a default label is the cleanest and most exam-appropriate solution.

Q195 Mixed Review: Cross-Topic MCQs *Difficult*

Which combination of constructs would correctly replicate switch-like exact-value branching using only if-else logic?

A. A single if with && chaining all values

B. An else-if ladder where each condition tests equality (==) against one specific value

C. A ternary operator nested infinitely

D. It is impossible to replicate switch using if-else

Ans: B

Explanation: An else-if ladder where each branch tests exact equality (x == value) against a specific constant can replicate the exact-match behavior of a switch statement, though less efficiently.

Q196 Mixed Review: Cross-Topic MCQs *Previous-Year Pattern*

Which of the following statements about decision-control structures in C is FALSE?

A. if-else allows exactly one of two blocks to execute

B. switch requires the controlling expression to be of integral type

C. The ternary operator can execute multiple independent statements per branch, just like if-else

D. else-if ladder tests conditions sequentially and stops at the first true condition

Ans: C

Explanation: This statement is FALSE: the ternary operator only returns a single value/ expression per branch and cannot execute multiple independent statements, unlike if-else blocks.

Q197 Mixed Review: Cross-Topic MCQs *Conceptual*

Why can compound relational tests sometimes replace nested if statements?

A. Because they always execute faster on every compiler

B. Because combining conditions with && can express the same 'all conditions must hold' logic that nested if achieves through hierarchy

C. Because C does not allow nested if statements

D. Because relational operators cannot be combined

Ans: B

Explanation: When every nested condition simply needs to be true together (with the same resulting action), a single compound && expression can express the same logic as multiple nested if levels.

Q198 Mixed Review: Cross-Topic MCQs *Application-Oriented*

Which control structure combination would best implement: 'If it is a weekday, check further whether it is a public holiday to decide if office is open'?

A. A single switch statement only

B. Nested if statements (outer: weekday check, inner: holiday check)

C. Ternary operator only

D. Logical NOT operator alone

Ans: B

Explanation: Since the holiday check is only relevant after confirming it is a weekday (a hierarchical, dependent decision), nested if statements correctly model this logic.

Q199 Mixed Review: Cross-Topic MCQs *Comparison-Based*

In terms of code readability for grouping several case values that share identical behavior, which is more suitable — switch fall-through or a long else-if chain with OR conditions?

A. Switch fall-through (grouping case labels without break) is generally cleaner for many discrete shared values

B. else-if chain with OR is always cleaner regardless of value count

C. Both are equally unreadable in all situations

D. Neither construct can group shared behavior

Ans: A

Explanation: When grouping many discrete constant values that share the same action, stacking case labels with fall-through in switch is typically more concise and readable than a long chain of OR'd equality checks.

Q200 Mixed Review: Cross-Topic MCQs *Assertion-Reason*

Assertion (A): Logical operators are essential for building compound relational tests.
Reason (R): && and || allow multiple individual relational comparisons to be merged into one combined condition.
Choose the correct option:

A. Both A and R are true, and R correctly explains A

B. Both A and R are true, but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: A

Explanation: Compound relational tests are defined precisely as multiple relational expressions joined by logical operators, so R directly and correctly explains why A is true.